

TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

SAĞLIKTA YAPAY ZEKÂ YARIŞMALARI

(Bilgisayarlı Görüyle Abdomen (Karın) Bölgesi için
Hastalık Tespiti Kategorisi)

PROJE DETAY RAPORU



TAKIM ADI: TEAM HAXIS

Team Haxis-AI v1

TAKIM ID: 468238

İÇİNDEKİLER

Proje Mevcut Durum Değerlendirmesi	3
Özgünlük	7
Sonuçlar ve İnceleme	8
Deney ve eğitim aşamalarında kullanılan veri setleri	10
Referanslar	11



1. Proje Mevcut Durum Değerlendirmesi

Projemizi büyük ölçekte PSR’de belirttiğimiz şekilde geliştirmekteyiz. Geliştirme sürecine paylaşılan veri setini anlamak ve analiz etmek ile başladık. Veri seti üzerinde yaptığımız analizler sonucunda dikkatimizi çeken birkaç durum oldu.

Bu durumlardan birincisi bizden istenen BBox(Sınırlayıcı Kutu) değerlerinin dikdörtgen prizması olarak değil, her bir kesit için ayrı ayrı dikdörtgenler şeklinde olduğunu fark ettik. Bu sebeple kullanacağımız modelin 3D(3 boyutlu) olması yerine 2D(2 boyutlu) olmasında karar verdik. Bu sayede modelimiz her bir kesiti ayrı ayrı işleyerek istenen BBox değerini bulabilecektir.

Yaptığımız analizlerde fark ettiğimiz diğer bir durum ise görüntülerin farklı boyutlarda ve farklı pencere aralıklarında olmasıydı. Konvolüsyon katmanlarının çalışabilmesi için verilen girdilerin boyutlarının sabit olması gerekmektedir. Bu sorunu çözmek için görüntülere “Resizing”(Yeniden Boyutlandırma) işlemi uygulanarak (800x800) boyutlarına getirilmiştir. “Resizing” işleminin örnek gösterimi Görsel 1’de belirtilmiştir. Görüntülerin farklı pencere aralıklarında olması ise modelimizin başarısını çok büyük ölçüde düşürüyordu. Görüntülerin pencere aralıkları sabitlenerek ve değer aralığı [0,1] olarak ayarlanarak bu sorun da çözülmüştür.



Görsel 1: “Resizing” işleminin örnek gösterimi

Veri dosyasını eğitimde kullanılabilecek şekilde düzenledikten sonra modelimizi geliştirmeye başladık. Modelimizi geliştirirken yaşanan sorunları daha rahat analiz edebilmek ve yapılacak olan değişiklikleri daha kolay bir şekilde uygulayabilmek için modelimizi kendi içinde eğitilebilir 3 ayrı modüle ayırdık. Bu modüllerin isimleri sırasıyla: “CNN”, “RPN” ve “R-CNN”dir.

CNN modülü VGG-16 mimarisine çok benzer bir şekilde tasarlanmıştır ve diğer modüllerde girdi olarak kullanılacak olan feature mapleri(özellik haritaları) oluşturur. CNN modülünün katmanları 6 bölüme ayrılmıştır. Son bölüm hariç her bölüm Conv2D, Batch Normalization, Max Pooling ve Dropout katmanlarından oluşmaktadır. Son bölüm ise Conv2D ve Batch Normalization katmanlarının yanında Flatten ve Dense katmanlarını da içermektedir. Tüm konvolüsyon katmanlarının kernel boyutu (3x3), aktivasyon fonksiyonu “ReLU”; Pooling katmanlarının havuz boyutu (2x2); Dropout katmanlarının rate(oran) değeri ise 0.2 olarak belirlenmiştir. Son katman olan Dense katmanının aktivasyon fonksiyonu sınıflandırma yapabilmesi için “Softmax” olarak seçilmiştir. Modülün loss fonksiyonu “Categorical Cross Entropy”, optimizasyon algoritması ise “RMSprop” olarak belirlenmiştir. CNN modülündeki Dense katmanı yarışma için işlevsizdir fakat modülün eğitiminde ve yapılan testlerde kullanılmıştır. CNN modülünün son katmanlarında bulunan “batch_normalization_8” ve “batch_normalization_9” katmanlarından alınan feature map çıktıları diğer modüllere iletilmektedir. CNN modülünün katman yapısı Görsel 2 ve Görsel 3’te belirtilmiştir(Görseller birbirinin devamı niteliğindedir).

Layer (type)	Output Shape	Param #	
conv2d (Conv2D)	(None, 800, 800, 16)	160	Bölüm 1
batch_normalization (Batch Normalization)	(None, 800, 800, 16)	64	
max_pooling2d (MaxPooling2D)	(None, 400, 400, 16)	0	
dropout (Dropout)	(None, 400, 400, 16)	0	
conv2d_1 (Conv2D)	(None, 400, 400, 32)	4640	Bölüm 2
batch_normalization_1 (Batch Normalization)	(None, 400, 400, 32)	128	
conv2d_2 (Conv2D)	(None, 400, 400, 32)	9248	
batch_normalization_2 (Batch Normalization)	(None, 400, 400, 32)	128	
max_pooling2d_1 (MaxPooling2D)	(None, 200, 200, 32)	0	Bölüm 3
dropout_1 (Dropout)	(None, 200, 200, 32)	0	
conv2d_3 (Conv2D)	(None, 200, 200, 64)	18496	
batch_normalization_3 (Batch Normalization)	(None, 200, 200, 64)	256	
conv2d_4 (Conv2D)	(None, 200, 200, 64)	36928	
batch_normalization_4 (Batch Normalization)	(None, 200, 200, 64)	256	
max_pooling2d_2 (MaxPooling2D)	(None, 100, 100, 64)	0	
dropout_2 (Dropout)	(None, 100, 100, 64)	0	

Görsel 2 : “CNN” modülünün katman yapısı

conv2d_5 (Conv2D)	(None, 100, 100, 128)	73856	Bölüm 4
batch_normalization_5 (Batch Normalization)	(None, 100, 100, 128)	512	
conv2d_6 (Conv2D)	(None, 100, 100, 128)	147584	
batch_normalization_6 (Batch Normalization)	(None, 100, 100, 128)	512	
max_pooling2d_3 (MaxPooling2D)	(None, 50, 50, 128)	0	Bölüm 5
dropout_3 (Dropout)	(None, 50, 50, 128)	0	
conv2d_7 (Conv2D)	(None, 50, 50, 256)	295168	
batch_normalization_7 (Batch Normalization)	(None, 50, 50, 256)	1024	
conv2d_8 (Conv2D)	(None, 50, 50, 256)	590080	Bölüm 6
batch_normalization_8 (Batch Normalization)	(None, 50, 50, 256)	1024	
max_pooling2d_4 (MaxPooling2D)	(None, 25, 25, 256)	0	
dropout_4 (Dropout)	(None, 25, 25, 256)	0	
conv2d_9 (Conv2D)	(None, 25, 25, 512)	1180160	
batch_normalization_9 (Batch Normalization)	(None, 25, 25, 512)	2048	
flatten (Flatten)	(None, 320000)	0	
dense (Dense)	(None, 7)	2240007	

Görsel 3: “CNN” modülünün katman yapısı

RPN modülü eğitilirken ilk olarak görüntü üzerinde 16 pixel aralıklarla noktalar oluşturur. Bu oluşturulan noktalar anchorların(pencere) konumlandırılacağı yerlerdir ve toplam 2500(yatayda ve dikeyde 800/16=50 tane toplam 50x50=2500) tanedir. Anchorlar 3 farklı en-boy oranlarında(0.5, 1, 2) ve boyuttur yani toplam 9(3x3=9) çeşit anchor vardır. RPN modülü bu anchorları oluşturduğu her bir noktanın üzerine yerleştirir ve toplam 22500(2500x9=22500) adet anchor oluşmuş olur. Bu anchorlardan görüntü dışına çıkanların silinmesiyle geriye sadece 8940 anchor kalmış olur.

Geriye kalan anchorların veri setinde görüntü için paylaşılan GT BBox'larla(Ground Truth Bounding Box) olan IoU değerleri hesaplanır. Anchorlardan IoU değerleri 0.6 değerinden büyük olanların etiketi 1(pozitif), 0.3 değerinden küçük olanların etiketi 0(negatif) olarak belirlenir. IoU değeri bu iki sınırın arasında kalanlar ise gereksiz olarak nitelendirilir ve silinir. Anchorlardan değeri 1 ve 0 olanların sayısını 128'e eşitlemek(toplam 256) için rastgele şekilde bazı anchorlar iptal edilir. Bu işlemler tamamlandıktan sonra RPN modülünün eğitimi için gerekli veriler hazırlanmış olur.

RPN modülünün katmanları çok basittir. İlk katmanı (3x3) kernel boyutu olan bir konvolüsyon katmanıdır. Sonrasında katmanlar iki kola ayrılır. Bu iki kolda birer tane (1x1) kernel boyutu olan Konvolüsyon katmanı vardır. Bu katmanlardan bir tanesi "linear" aktivasyon fonksiyonuna sahiptir ve patoloji olabilecek BBox değerlerini tahmin eder. Diğer katman ise "sigmoid" aktivasyon fonksiyonuna sahiptir bu sayede modül, BBox'larda patoloji olma durumunu tahmin edebilir. RPN modülünün katman yapısı Görsel 4'te paylaşılmıştır.

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 27, 27, 512)]	0	[]
fmapconvolution (Conv2D)	(None, 25, 25, 512)	2359808	['input_1[0][0]']
scores1 (Conv2D)	(None, 25, 25, 9)	4617	['fmapconvolution[0][0]']
deltas1 (Conv2D)	(None, 25, 25, 36)	18468	['fmapconvolution[0][0]']

Görsel 4: "RPN" modülünün katman yapısı

RPN modülü eğitilirken girdi olarak CNN modülünden alınan feature mapler verilir. Çıktı olarak ise hesaplanan ve etiketlenen anchorlar beklenir. RPN modülünde optimizasyon algoritması olarak "Adam" kullanılmış ve loss fonksiyonunun matematiksel formülü Görsel 5'te belirtilmiştir.

$$L(p_i, t_i) = \underbrace{\frac{1}{N_{cls}} \sum_i L_{cls}(p_i, p_i^*)}_{\text{object/not object classifier}} + \lambda \underbrace{\frac{1}{N_{reg}} \sum_i p_i^* L_{reg}(t_i, t_i^*)}_{\text{box regressor}}$$

p_i : predicted probability

$p_i^* \begin{cases} 1 \text{ for pos anchor} \\ 0 \text{ for neg anchor} \end{cases}$

N_{cls} : numbers of anchors in minibatch (512)

λ = constant value

N_{reg} = number of total anchors (~2000)

$p_i^* \begin{cases} 1 \text{ for pos anchor} \\ 0 \text{ for neg anchor} \end{cases}$

$$\begin{aligned} t_x &= (x - x_a)/w_a, & t_y &= (y - y_a)/h_a, \\ t_w &= \log(w/w_a), & t_h &= \log(h/h_a), \\ t_x^* &= (x^* - x_a)/w_a, & t_y^* &= (y^* - y_a)/h_a, \\ t_w^* &= \log(w^*/w_a), & t_h^* &= \log(h^*/h_a), \end{aligned}$$

$$L_{reg} = \text{smooth}_{L_1}(t_i - t_i^*) \quad \text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

t_i = predicted box; t_i^* = ground truth box

Görsel 5: RPN modülünün loss fonksiyonunun matematiksel formülü

R-CNN modülünün eğitilmesi için ilk olarak RPN modülünden gelen BBox çıktıları alınır ve bu çıktıların NMS(Non Maximum Suppression) uygulanır. NMS aynı bölge için oluşturulmuş birden fazla BBox'ı tek bir BBox haline getirmek için kullanılır. NMS'nin örnek gösterimi Görsel 6'te belirtilmiştir.



Görsel 6: NMS'nin örnek gösterimi

Çıktılara NMS uygulandıktan sonra kalan işlemler RPN modülüne çok benzemektedir. Alınan BBox çıktıların veri setinde paylaşılan GT BBox'larla olan IoU değerleri hesaplanır. Bu BBox'lardan IoU değeri 0.5'in üzerinde olanlar GT BBox'ın sahip olduğu patoloji etiketi ile etiketlenir(1-6). IoU değeri 0.2'den küçük olanlar ise 0(patoloji yok) ile etiketlenir. Patoloji olan ve olmayan BBox'ların sayısının eşitlenmesi için bazı BBox'lar rastgele şekilde iptal edilir.

R-CNN katmanlarının ilk bölümü Flatten, Dense ve ardından gelen Batch Normalization katmanından oluşur. Sonrasında katmanlar iki kola ayrılır. Bu kolların ilki verilecek olan BBox çıktısını tahmin etmek için 4 düğümü ve "linear" aktivasyon fonksiyonu kullanan bir Dense katmanından oluşur. Diğer kolu ise patoloji sınıfını tahmin etmek için 7 düğümlü, aktivasyon fonksiyonu "Softmax" olan bir Dense katmanı kullanır. R-CNN modülünün katman yapısı Görsel 7'te paylaşılmıştır.

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[None, 7, 7, 256]	0	[]
flatten (Flatten)	(None, 12544)	0	['input_1[0][0]']
fc2 (Dense)	(None, 1024)	12846080	['flatten[0][0]']
batch_normalization (Batch Normalization)	(None, 1024)	4096	['fc2[0][0]']
deltas2 (Dense)	(None, 4)	4100	['batch_normalization[0][0]']
scores2 (Dense)	(None, 7)	7175	['batch_normalization[0][0]']

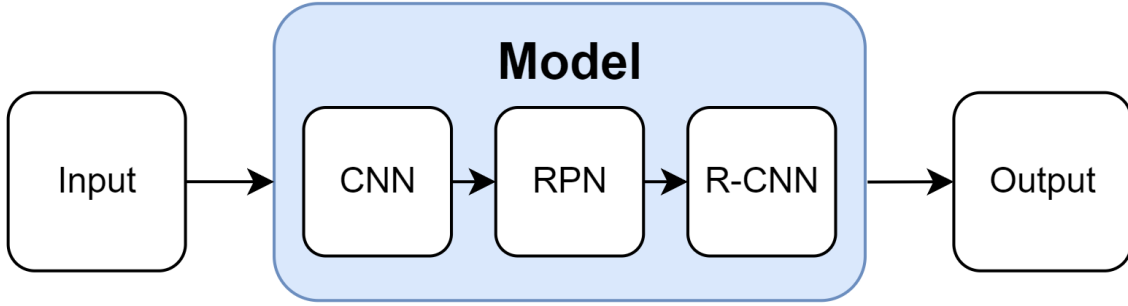
Görsel 7: "RCNN" modülünün katman yapısı

R-CNN modülü eğilirken CNN modelinden gelen feature mapler hesaplanan BBox değerlerine göre kırılır ve modüle girdi olarak verilir. Modelin çıktısı olarak ise veri setinde paylaşılan GT BBox'lar ve sınıf etiketleri beklenir. Modelin optimizasyon algoritması "adam"; sınıflandırma kolunun loss fonksiyonu "Categorical Cross Entropy", BBox kolunun loss fonksiyonu ise "huberGTZ" olarak belirlenmiştir.

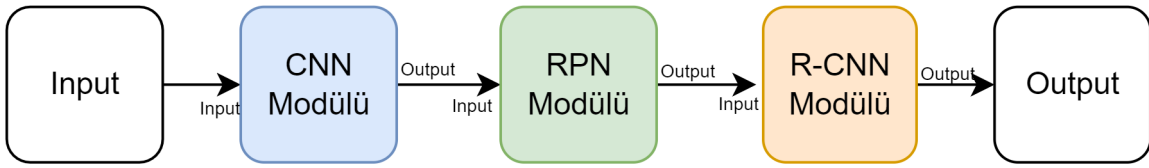
R-CNN modülünden alınan koordinat çıktıları görüntülere uyguladığımız "Resizing" işleminin tersi uygulanarak veri setinde belirtilen değerlere ulaşılmaktadır.

2. Özgünlük

Eğitimde kullanılacak veri setinin büyüklüğü ve modelin eğitileceği cihazın özellikleri dikkate alındığında eğitim süresinin çok uzun olacağı görülmüştür. Genellikle kullanılan tek model tasarımında, model üzerinde yapılan değişiklikler sonrası modelin en baştan tekrardan eğitilmesi gerekmektedir. Bu sorunu çözmek için kullanılacak model 3 ayrı modüle ayrılmıştır. Bu modüller kendinden önce gelen modül(ler)e bağlı olarak ayrı ayrı eğitilebilmektedir. Bu sayede sadece değişiklik yapılan modül ve ardından gelen modül(ler) tekrardan eğitilerek eğitim süresinden kazanç sağlamak amaçlanmıştır. Tek model tasarımının örnek gösterimi Görsel 8’te, çok modüllü tasarımın örnek gösterimi Görsel 9’te belirtilmiştir.



Görsel 8: Tek model tasarımının örnek gösterimi



Görsel 9: Çok modüllü tasarımın örnek gösterimi

Modelimizin bu şekilde tasarlanmasının tek amacı geliştirme sürecinde bize sağladığı eğitim süresini kısaltmak değildir. Modelimizdeki hataları bulmak, verilen çıktıkları kolay bir şekilde incelemek ve performans analizlerini yapmak gibi konularda da çokça avantajı bulunmaktadır. Modelimizin bu özelliği özgünlüğe katkı sağlamaktadır.

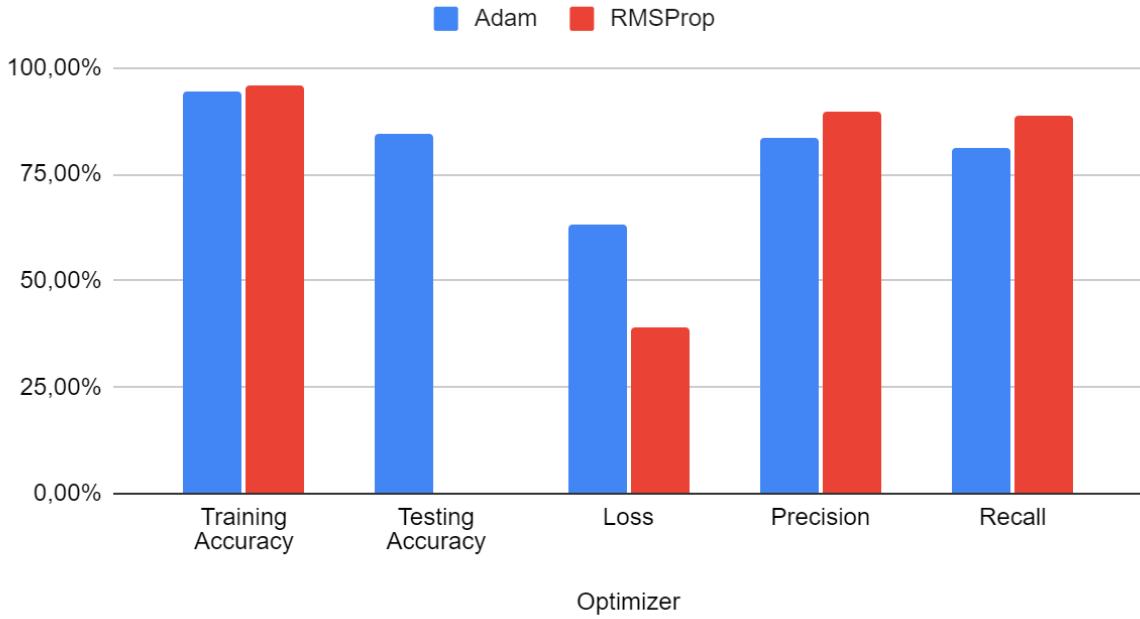
Modelimizin CNN modülünde kullandığımız VGG-16 mimarisi yapısındaki Conv2D ve Pooling(Havuzlama) katmanları sayesinde modelimize en uygun boyutlarda ve yeterli sayıda feature mapler oluşturmaktadır. Bu mimariye eklediğimiz “Batch Normalization” ve “Dropout” katmanları ile modülümüzün başarı oranı artırmak ve Overfitting(aşırı öğrenme) durumunun önüne geçilmek amaçlanmıştır. VGG-16 mimarisinin örnek gösterimi Görsel 10’da belirtilmiştir.



Görsel 10: VGG-16 mimarisinin örnek gösterimi

Modelimizin; CNN modülünde “RMSprop”, RPN ve R-CNN modülünde ise “Adam” optimizasyon algoritmaları kullanılmıştır. “RMSprop” konvolüsyon katmanlarında daha iyi sonuçlar verdiği için CNN modülünde kullanılmıştır[4]. Tablo 1’de “RMSProp” ve “Adam” optimizasyon algoritmalarının konvolüsyon katmanları üzerindeki performans karşılaştırması verilmiştir.

Adam vs RMSProp



Tablo 1: “RMSProp” ve “Adam” optimizasyon algoritmalarının konvolüsyon katmanları üzerindeki performans karşılaştırması

Modelimizin “RPN” ve “R-CNN” katmanlarında optimizasyon algoritması olarak “Adam” kullanılmıştır. Bu modüllerde “Adam” kullanılmasının sebebi algoritmanın sınıflandırma problemlerindeki yüksek başarı oranıdır. Sınıflandırma problemlerinde Adam’ın başarı oranı %92,31, “RMSProp”’un başarı oranı ise %40,26 olarak gözlemlenmiştir[5]. Sınıflandırma performansında büyük bir fark olmasına rağmen CNN modülünde “RMSProp” kullanma sebebimiz, CNN modülündeki sınıflandırmanın yarışma için işlevsiz olmasıdır.

Modelimizde tek bir optimizasyon algoritması kullanmak yerine farklı modüllerde farklı optimizasyon algoritmaları kullanmak özgünlüğe katkı sunmuştur.

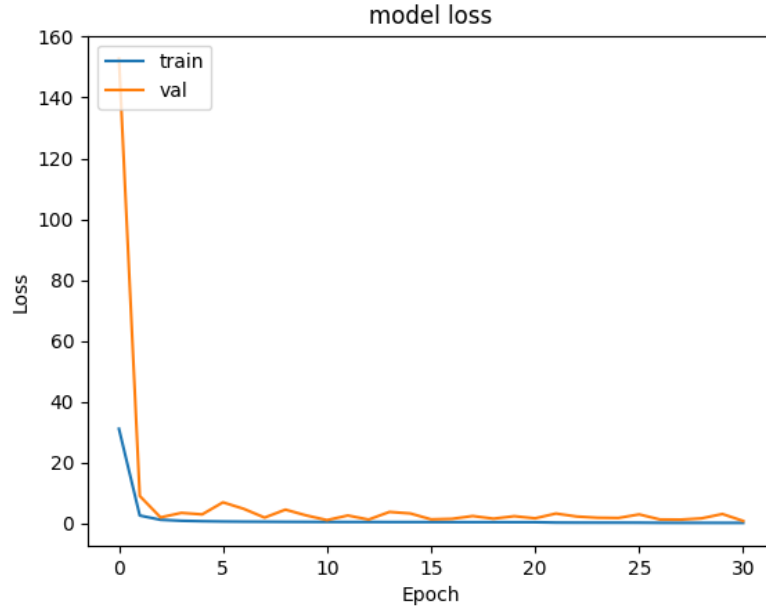
3. Sonuçlar ve İnceleme

Yapılan eğitim ve testler PSR’de belirtilen “Asus Rog Strix G513QM-HQ358” cihazından yapılmıştır. Modelimizi 3 ayrı modüle ayırdığımız için modüller tek tek eğitilmiştir. Eğitim ve test sürecinde yaşadığımız sorunlar ve elde ettiğimiz veriler şu şekildedir:

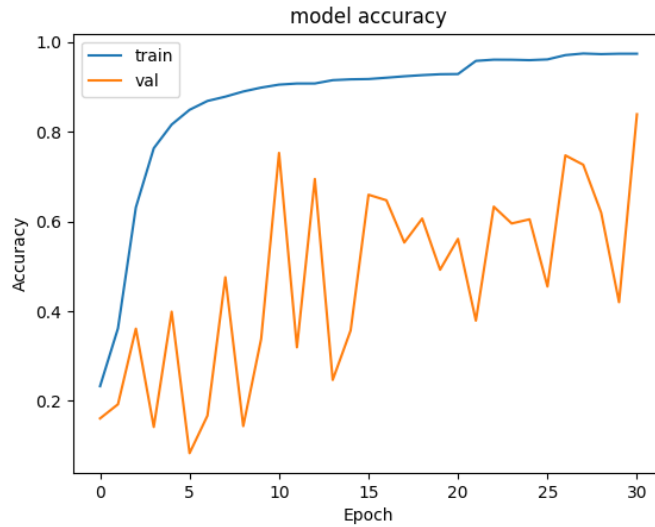
CNN modülünde eğitim ve test için kullanılacak olan kesitler yanlış öğrenme durumu olmaması için patoloji içermeyen ve sadece bir tane patoloji içeren kesitler arasından seçilmiştir. Test için her durumdan eşit sayıda veri olmak şartıyla toplam 994 adet test verisi rastgele şekilde ayrılmıştır. Eğitim için “Batch Size” 8, “Epoch” 30 ;modülün overfit olmasını engellemek amacıyla “Learning Rate” ilk 20 epoch $1e-3$, sonraki 5 epoch $5e-4$, son 5 epoch için ise $3e-4$ olarak belirlenmiştir.

Modelin ilk eğitiminden sonra test sonuçlarında %20 civarında bir accuracy(doğruluk) elde edilmiştir. Accuracy değerinin bu kadar düşük olmasının sebebini araştırdığımızda; bunun sebebini görüntülerdeki pencere aralığının farklı olmasından veya modelin her epoch sonrasında kaydedilme koşulu loss(eğitim aşamasında elde edilen loss değeri) değerinin düşmesi olarak belirlenmesinden kaynaklanabileceğini düşündük. Görüntülerin pencere aralıkları sabitlendikten ve kaydetme koşulu “val_loss”(test aşamasında elde edilen loss

değeri) değerinin düşmesi olarak belirlendikten sonra tekrar eğitim sürecine başlandı. Eğitim ve test sürecine ait loss ve accuracy değerleri sırası ile Görsel 11 ve Görsel 12’de belirtilmiştir.



Görsel 11: Eğitim ve test sürecine ait loss değerleri



Görsel 12: Eğitim ve test sürecine ait accuracy değerleri

CNN modülü eğitildikten sonra RPN modülünün eğitilmesi gerekmektedir. RPN modülünün eğitiminde veri setinde patoloji içeren her kesit rastgele şekilde kullanılmıştır. Eğitim için “Batch Size” 256, “Steps” 300, “Epochs” 20 olarak belirlenmiştir. “Learning Rate” değeri için kullanılan fonksiyon Görsel 13’te belirtilmiştir .

$$f(Epoch) = 0.001 \times (1 - 0.9 \times Epoch / 20)$$

Görsel 13: RPN modülünün Learning Rate değeri için kullanılan fonksiyon

Eğitim bittikten sonra CNN modülü testinde de olduğu gibi 994 adet rastgele test verisi ile test yapılmıştır. Test sonucunda elde edilen loss değerleri Tablo 2’de belirtilmiştir.

Total Loss	Score Loss	Delta Loss
0,373	0,364	0,009

Tablo 2: RPN test sonucunda elde edilen loss değerleri
Score Loss patoloji olma durumu, Delta Loss ise BBox koordinatları ile ilgilidir.

R-CNN modülü eğitilirken veri setinde patoloji içeren her kesit ve patoloji içermeyen durumlardan da 9000 adet kullanılmıştır. Eğitimde “Batch Size” 256, “Steps” 300, “Epochs” 5, “Learning Rate” ise $2e-4$ olarak belirlenmiştir. Eğitim sonunda modeli test ettiğimizde aldığımız çıktıların büyük çoğunluğu alakasız veya yanlıştır. R-CNN modülündeki bu sorun sebebiyle sonuçları paylaşamamaktayız.

CNN modülünün test grafiğindeki dalgalanmalardan ve R-CNN modülündeki hatalı çıktılardan yola çıkarak bu modüllerimizin Overfit olmuş olabileceğini düşünmekteyiz. CNN modülünün sadece konvolüsyon katmanlarını kullanmamız ve çok kötü bir değer olmayan %84 accuracy oranı almamız bu sorunu R-CNN modülündeki soruna göre daha az önemli yapmaktadır. R-CNN modülünün eğitim süresi çok uzun olduğundan rapor teslim tarihine kadar sorunun çözülmesi ve modelin tekrardan eğitilmesi mümkün olmamıştır fakat sorunu çözmek için çalışmalarımıza tüm hızıyla devam etmekteyiz.

4. Deney ve eğitim aşamalarında kullanılan veri setleri

Projemizde veri seti olarak sadece Sağlık Bakanlığı tarafından paylaşılan veri seti kullanılmıştır. Sağlık Bakanlığı içinde DICOM formatında seriler ve serilerdeki patolojilerle ilgili EXCEL formatında bir veri dosyasını ZIP formatında sıkıştırılmış olarak paylaşmıştır. Patolojilerin hangi kesit aralığında görülebileceği, var olan patolojilerin BBox değerleri ve sınıfı paylaşılan veri dosyasında belirtilmiştir. Sağlık Bakanlığı tarafından paylaşılan veri setinde BBox koordinatlarıyla birlikte paylaşılan patoloji miktarı Tablo 3’te belirtilmiştir.

Şartname Sınıfı	Patoloji Miktarı
1	5559
2	4925
3	6923
4	2556
5	1148
6	9817

Tablo 3: Sağlık Bakanlığı tarafından paylaşılan veri setindeki patoloji miktarı

Veri seti analizi sırasında veri setinde birkaç istisna dışında bir seri içindeki tüm kesitlerin boyutlarının aynı olduğu anlaşılmıştır. Kesitlerin boyutları çoğunlukla 512x512 olmakla birlikte farklı serilerde bu boyutun değişebileceği gözlemlenmiştir. Bu sebeple görüntüler kullanılmadan önce en uygun değer olarak görülen (800x800) boyutuna getirilmiştir.

Veri dosyasında paylaşılan veriler “Olgu Numarası” ve “Kesit Numarası”na göre sıralanmıştır. Dolayısıyla veriler sınıf numaralarına göre gruplanarak kullanılmıştır.

Sağlık Bakanlığının paylaştığı veri setinde yapılan analizler sonucunda veri seti yeterli görülmüş ve harici bir veri setine ihtiyaç duyulmamıştır. Sonuç olarak deney ve eğitim aşamalarında sadece Sağlık Bakanlığı tarafından paylaşılan veri seti kullanılmıştır.

5. Referanslar

[1] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks“, [arXiv:1506.01497](https://arxiv.org/abs/1506.01497), 2016.

[2] [3. How RPN \(Region Proposal Networks\) Works - YouTube](https://www.youtube.com/watch?v=3Hh3p3p3p3p)

[3] [Non Maximum Suppression: Theory and Implementation in PyTorch \(learnopencv.com\)](https://www.learnopencv.com/non-maximum-suppression-theory-and-implementation-in-pytorch/)

[4] B. Soujanya and T. Sitamahalakshmi, “Optimization with ADAM and RMSprop in Convolution neural Network (CNN)“, [warse.org](https://www.warse.org), 2020

[5] Ebubekir SEYYARER , Faruk AYATA , Taner UÇKAN ve Ali KARCI, “Derin Öğrenmede Kullanılan Optimizasyon Algoritmalarının Uygulanması Ve Kiyaslanması“, dergipark.org.tr, 2020

